# Low-Latency Concept Drift Detection and Online Adaptation for High-Throughput Streaming Classification

Yash Patidar
*School of Computer Engineering*
*Manipal Institute of Technology,*
*Manipal Academy of Higher Education*
Manipal, Karnataka
yash8.mitmpl2024@learner.manipal.edu

Pragun Kakkar
*School of Computer Engineering*
*Manipal Institute of Technology,*
*Manipal Academy of Higher Education*
Manipal, Karnataka
pragun.mitmpl2024@learner.manipal.edu

Shantanu Singh
*School of Computer Engineering*
*Manipal Institute of Technology,*
*Manipal Academy of Higher Education*
Manipal, Karnataka
shantanu.mitmpl2024@learner.manipal.edu

***Abstract*—Continuous, high-velocity data streams generated by IoT sensor networks and real-time transaction pipelines frequently suffer from concept drift, where the underlying statistical distribution shifts over time and degrades predictive accuracy. Existing drift detection frameworks often incur heavy computational overhead or rely on static thresholds that produce high false-alarm rates during gradual, non-stationary shifts. In this work, we present a lightweight concept drift detection and online adaptation framework for high-throughput streaming classification. The detector is implemented from first principles as a one-sided Page-Hinkley cumulative-sum test over the classifier's binary error signal, with a bias-corrected Exponentially Weighted Moving Average (EWMA) baseline; it runs in $O(1)$ time and memory per sample without sliding-window buffers. We integrate it into an interleaved test-then-train (prequential) pipeline with two incremental classifiers from `river`: a Hoeffding Adaptive Tree (HAT), which relies on its internal ADWIN-based subtree replacement, and a streaming Logistic Regression (LR) pipeline, which is hard-reset on every alarm. On a synthetic SEA stream with an abrupt concept change at sample 25,000, the detector raises exactly one alarm per model (delays of 1,213 and 1,356 samples) and no alarm during the 25,000 pre-drift samples; LR attains 99.3% overall prequential accuracy versus 97.1% for HAT, and its rolling accuracy stays at or above 0.95 from 1,837 samples after the drift (6,519 for HAT). On the real-world Elec2 benchmark, both models raise 10 alarms and reach practically identical overall prequential accuracy (83.7% for LR, 84.0% for HAT), while LR processes a sample in 10.61 $\mu$s on average versus 77.32 $\mu$s for HAT (7.3$\times$ faster; 6.2$\times$ on SEA). The results show that a linear learner paired with a constant-time drift detector and hard resets matches or exceeds the accuracy of an adaptive tree at a fraction of its per-sample cost.***

***Index Terms*—concept drift, online learning, data stream mining, EWMA, Page-Hinkley test, Hoeffding Adaptive Tree, prequential evaluation**

## I. Introduction and Motivation

Modern data-driven systems, including IoT sensor clusters, high-frequency financial platforms, and network intrusion monitors, process continuous streams of non-stationary data. In these operational environments, the underlying data-generating distribution changes over time due to external environmental variations, evolving user habits, or economic fluctuations. This phenomenon is known as *concept drift* [1]. When concept drift occurs, models trained on past data experience substantial degradation in predictive accuracy [3]. Addressing this requires online learning architectures capable of updating incrementally with each incoming instance.

However, deploying adaptive machine learning models on high-throughput streaming pipelines introduces strict resource constraints. A streaming classifier must process incoming records in real time under bounded per-sample execution latency and finite memory footprint [4]. Recent research has increasingly applied deep learning and representation-based methods to drift detection [4], [7]. While these techniques can identify complex distribution shifts, they introduce large architectural footprints, significant computational latencies, and buffering requirements that are ill-suited for microsecond-scale data ingestion [4]. Conversely, traditional lightweight statistical detectors often rely on fixed, static decision thresholds that trigger false alarms during natural, gradual variance shifts in stationary streams [6]. Furthermore, standard reactive detection paradigms often suffer an extended performance collapse before gathering enough historical error evidence to trigger model adaptation [8].

This paper evaluates an efficient middle ground: a lightweight, constant-time concept drift detection and online adaptation framework. We implement a Page-Hinkley cumulative-sum test whose reference level is an adaptive, bias-corrected EWMA of the classifier's error signal. The detector operates in $O(1)$ time and memory per sample, following slow changes in the baseline error rate while remaining sensitive to sustained increases. We couple this detector with two incremental classifiers—a Hoeffding Adaptive Tree and a streaming Logistic Regression pipeline—and evaluate them under the prequential protocol on a synthetic and a real-world benchmark.

The primary contributions of this work are as follows:

- An $O(1)$-time, $O(1)$-memory Page-Hinkley drift detector with a bias-corrected EWMA baseline, implemented from first principles (no dependency on `river.drift`) and

operating on the binary error stream without window buffers.
- A prequential evaluation pipeline with model-specific adaptation hooks, contrasting HAT's internal ADWIN-driven subtree replacement (the external alarm triggers no model change) with a hard reset of the scaler and LR model.
- An evaluation on SEA and Elec2 reporting per-sample rolling accuracy, overall prequential accuracy, detection delay, recovery behaviour, and per-sample wall-clock latency measured with `time.perf_counter_ns`.
- A verification of the reported results directly from the exported telemetry: the per-sample error sequence is reconstructed from the logged rolling accuracy, and an offline replay of the detector reproduces all 22 logged alarm positions exactly.

The remainder of this report is organized as follows. Section II surveys recent literature on concept drift detection. Section III identifies the research gap, Section IV states the formal problem, and Section V lists the project objectives. Section VI describes the datasets and preprocessing procedures. Section VII details the proposed methodology and system architecture. Section VIII outlines the implementation details. Section IX presents the experimental results. Section X discusses the findings, strengths, limitations, and alignment with project objectives. Section XI concludes the paper with directions for future scope.

## II. Literature Survey and Comparative Analysis

The literature on concept drift detection reflects an ongoing trade-off between computational overhead and detection sensitivity. Early statistical tests provided lightweight solutions but lacked adaptability to gradual distribution shifts. In recent years, unsupervised representation learning and deep neural networks have gained popularity. For example, Wan et al. [4] introduced Maximum Concept Discrepancy (MCD) via contrastive learning to identify drift in high-dimensional spaces without supervision. Chambers et al. [7] proposed AdaDeepStream, utilizing intermediate CNN activations to detect drift with limited true labels. While effective in complex domains, deep approaches incur computational costs that create severe latency bottlenecks in high-velocity streams.

To reduce supervisory overhead, Fellicious et al. [10] developed SUDS, an unsupervised drift sampling technique that selects homogeneous samples for model retraining, reducing annotation requirements by up to 80%. Wu and Xiong [2] explored online incremental parameter updates in Bayesian classifiers for non-stationary environments, achieving low computational cost but showing sensitivity to class imbalance. Aguiar and Cano [5] applied meta-learning over sliding-window meta-features to dynamically select the most suitable drift detector, though this requires extensive offline meta-training. Mulimani et al. [1] combined ensemble learning with adaptive sliding windows in an online LightGBM framework, which performs well on gradual drift but responds sluggishly to abrupt changes.

To mitigate computational bottlenecks, recent research has returned to statistical and probabilistic formulations. Parasteh and Sadaoui [3] developed PRDD, a probabilistic detector tracking classifier confidence shifts in $O(1)$ time, demonstrating that tracking prediction error streams can yield high sensitivity without buffering raw data. Rum and Song [6] demonstrated that dynamically recalibrating reconstruction error thresholds using an EWMA baseline substantially suppresses false alarms in stationary and gradual drift streams. Finally, proactive adaptation mechanisms have emerged: Guerrero Cano et al. [8] developed proactive Hoeffding Tree variants estimating boundary trajectories to adapt before performance collapses, while Zhao and Shen [9] proposed PROCEED to proactively adjust model parameters for online time-series forecasting. Table I provides a comparative summary of these recent approaches.

## III. Research Gap

Existing research reveals two prominent gaps in streaming concept drift adaptation:

1) **High computational cost versus rigid thresholding:** Deep learning and representation-based methods [4], [7] isolate complex drifts effectively but introduce heavy computational latency and memory buffering that make them impractical for high-velocity streaming. Conversely, lightweight statistical detectors often rely on static validation thresholds that fail to adjust to long-term gradual drift, resulting in frequent false alarms [6].
2) **Reactive latency in online adaptation:** Most lightweight stream algorithms operate reactively, requiring a marked decline in predictive accuracy before accumulating sufficient statistical evidence to trigger an alarm [8]. There is a lack of compact, constant-time adaptation frameworks that dynamically scale their internal baselines to detect drifts with minimal delay while maintaining strict microsecond-level processing throughput.

## IV. Problem Statement

To design, implement, and evaluate a low-latency, resource-bounded concept drift detection and online adaptation framework for high-throughput data streams. The proposed system must identify non-stationary distribution shifts in streaming classification models and initiate online model adaptation in real time, minimizing post-drift accuracy degradation while operating under strict $O(1)$ per-sample memory and time complexity constraints for the detector.

## V. Project Objectives

The specific objectives of this project are:

- **Objective 1:** Implement a constant-time $O(1)$ sequential concept drift detector based on the Page-Hinkley cumulative-sum test, tracking classification error streams without sliding window buffers [3].
- **Objective 2:** Integrate an adaptive baseline using a bias-corrected Exponentially Weighted Moving Average (EWMA) to continuously recalibrate the expected error level and suppress false alarms triggered by statistical noise [6].

TABLE I
COMPARATIVE ANALYSIS OF RECENT CONCEPT DRIFT DETECTION METHODOLOGIES

| Year | Title & Authors | Proposed Methodology | Strengths | Limitations |
|---|---|---|---|---|
| 2023 | *Online Learning of Bayesian Classifiers...* [2] | Online incremental algorithm updating Bayesian parameters per sample [2]. | Computationally economical; handles nonstationary streams [2]. | Lacks mechanisms for multi-class imbalanced data [2]. |
| 2023 | *AdaDeepStream: Streaming Adaptation...* [7] | Uses CNN activations with streaming machine learning models for drift detection [7]. | Rapid adaptation with minimal true-labeled samples [7]. | High architectural overhead; depends on explicit drift detection [7]. |
| 2023 | *Enhancing Concept Drift Detection...* [5] | Exploits meta-features from sliding windows to dynamically recommend drift detectors [5]. | Highly robust to class imbalance without relying on fixed detectors [5]. | Requires offline training of meta-models on large datasets [5]. |
| 2024 | *Online Detection and Adaptation...* [1] | Online LightGBM combining ensemble learning and adaptive sliding windows [1]. | Adaptive to both real and virtual concept drifts [1]. | Struggles with sudden, abrupt drift scenarios [1]. |
| 2024 | *A Probabilistic Approach for...* [3] | PRDD algorithm tracking posterior probability shifts using adaptive confidence [3]. | Constant time complexity $O(1)$ per sample; highly efficient [3]. | Dependent on the base classifier's probability calibration [3]. |
| 2024 | *Online Drift Detection with...* [4] | Computes Maximum Concept Discrepancy (MCD) via contrastive learning [4]. | Unsupervised operation; effective on complex, high-dimensional data [4]. | Deep encoder optimization requires significant resources [4]. |
| 2024 | *SUDS: A Strategy for Unsupervised...* [10] | Unsupervised drift sampling to select homogeneous samples for retraining [10]. | Reduces annotated data requirement by up to 80% [10]. | Underperforms on artificial datasets lacking noise [10]. |
| 2025 | *PROCEED: Proactive Model Adaptation...* [9] | Maps latent concept drift to parameter adjustments proactively to overcome feedback delays [9]. | Anticipates drift before performance collapses [9]. | Relies on historical drift patterns; struggles with out-of-distribution shifts [9]. |
| 2026 | *An EWMA-Based Adaptive...* [6] | Dynamically updates reconstruction error thresholds using EWMA over autoencoders [6]. | Eliminates false alarms in stationary and gradual drift streams [6]. | Sensitive to tuning of the EWMA smoothing coefficient [6]. |
| 2026 | *Anticipating to Change...* [8] | Proactive Hoeffding Tree variants anticipating drift via boundary trajectory estimation [8]. | Prevents degradation proactively by maximizing class separability [8]. | Relies on linear boundary adjustments; struggles with erratic drifts [8]. |

- **Objective 3:** Couple the drift detector with incremental base learners (Hoeffding Adaptive Tree and streaming Logistic Regression) to execute online adaptation protocols upon receiving drift alarms [8].
- **Objective 4:** Evaluate the end-to-end framework under the prequential protocol on synthetic and real-world streaming benchmarks, quantifying detection delay, recovery behaviour, accuracy, and per-sample processing time [3], [6].

## VI. DATASET DESCRIPTION AND PREPROCESSING

The framework is evaluated on two standard streaming benchmarks: a synthetic stream with a single, controlled abrupt concept change and a real-world stream with naturally occurring, unlabelled drift.

### A. Synthetic SEA Concept Generator

The SEA generator of Street and Kim is used through `river.datasets.synth.SEA`. Each instance has three continuous attributes $f_1, f_2, f_3$ drawn uniformly from $[0, 10]$. The label is $y =$ True if $f_1 + f_2 > \theta$ and False otherwise; $f_3$ is irrelevant to the label. Label noise is disabled (`noise=0`). Two concepts are combined: `variant=0` ($\theta = 8$) before the change and `variant=1` ($\theta = 9$) after it, both generated with seed 42. The classes are therefore not balanced: 68.2% of the first 25,000 instances and 59.8% of the last 25,000 instances are positive. The change relabels only the instances with $8 < f_1 + f_2 \leq 9$ (8.4% of the post-change instances), so a model that kept the old boundary perfectly would see its error rate rise by about 8.4 percentage points.

The two concepts are mixed by `concept_drift_stream()` in `river_benchmarks.py`, which reproduces the sigmoid-weighted mixing of `river`'s `ConceptDriftStream` but evaluates the sigmoid in a numerically stable form (the library's direct evaluation of $1/(1 + e^v)$ overflows for a narrow width far from the change point). At sample $i$ the next instance is drawn from the post-change concept with probability $\sigma(4(i - p)/w)$, with position $p = 25{,}000$ and width $w = 1$, i.e. 0.018 at $i = 24{,}999$, 0.5 at $i = 25{,}000$ and 0.982 at $i = 25{,}001$; the change is effectively instantaneous: in the generated stream, samples 1–25,000 come from the first concept and samples 25,001–50,000 from the second. Because both generators share seed 42, the post-change segment replays the same feature vectors as the first 25,000 instances, relabelled under $\theta = 9$. The stream is truncated to $N = 50{,}000$ instances and processed strictly in order.

### B. Electricity Market Benchmark (Elec2)

Elec2, compiled by Harries (1999) from the Australian New South Wales (NSW) electricity market, is loaded with `river.datasets.Elec2`. It contains 45,312 instances recorded at 30-minute intervals between May 1996 and December 1998. Each instance has eight attributes: date, day of week (integer 1–7), half-hour period, NSW price, NSW demand, Victoria (VIC) price, VIC demand, and scheduled NSW–VIC transfer. In the distributed version used by `river`, all attributes except the day of week are already min–max

normalized to $[0, 1]$. The target indicates whether the NSW price went UP (42.5%) or DOWN (57.5%) relative to a moving average of the previous 24 hours; `river` maps UP to True. Elec2 contains naturally occurring concept drift (demand, regulatory, and seasonal changes), but there is no ground truth for when drift occurs. The full stream is processed in its chronological order without filtering or reordering.

### C. Preprocessing and Feature Engineering

All preprocessing is performed online:

- **Streaming standardization (LR only):** Features pass through `river.preprocessing.StandardScaler`, which maintains running estimates of each feature's mean and variance and transforms each instance to approximately zero mean and unit variance.
- **No scaling for HAT:** The tree is used on the raw attribute values; its split decisions depend only on the ordering of attribute values and are therefore invariant to monotonic rescaling.
- **Targets:** Both loaders already yield boolean labels (SEA: $f_1 + f_2 > \theta$; Elec2: UP), so no further encoding is applied.
- **No feature engineering:** No interaction terms, lag features, or polynomial expansions are used; all models receive exactly the attributes produced by the loaders (including Elec2's date attribute). Differences in recovery and accuracy therefore stem from the learners and their adaptation mechanisms rather than from handcrafted features.

## VII. Proposed Methodology and System Architecture

### A. System Architecture and Pipeline Workflow

The pipeline is the interleaved test-then-train (prequential) loop implemented by `evaluate()` in `stream_evaluation.py` and illustrated in Fig. 1. For each labelled instance $(x_t, y_t)$ it executes five stages, in this order:

1) **Prequential prediction:** The current model outputs $\hat{y}_t = f(x_t)$ before it is trained on $y_t$.
2) **Metric and error update:** The rolling accuracy and cumulative F1 score are updated with $(y_t, \hat{y}_t)$, and the binary error $e_t = \mathbf{1}[\hat{y}_t \neq y_t]$ is computed.
3) **Incremental learning:** The model is updated with $f.\text{LEARN}(x_t, y_t)$. The per-sample latency timer, started before stage 1, is stopped here.
4) **Drift monitoring:** $e_t$ is passed to the EWMA-PH detector, which updates its state in $O(1)$ operations.
5) **Adaptation:** If $PH_t > \lambda$, the alarm index $t$ is recorded, the model-specific callback $\text{ADAPT}(f)$ is invoked, and the detector is reset. Because learning precedes monitoring, the instance that triggers an alarm has already been learned by the pre-reset model, and a reset model starts learning from $x_{t+1}$.

### B. Adaptive Drift Detection via EWMA-PH

The drift monitor (`PageHinkleyDetector` in `page_hinkley.py`) is a Page-Hinkley cumulative-sum change-point test whose reference level is an exponentially weighted running mean, implemented from first principles. It consumes the scalar error signal $e_t \in \{0, 1\}$ through the following recurrences.

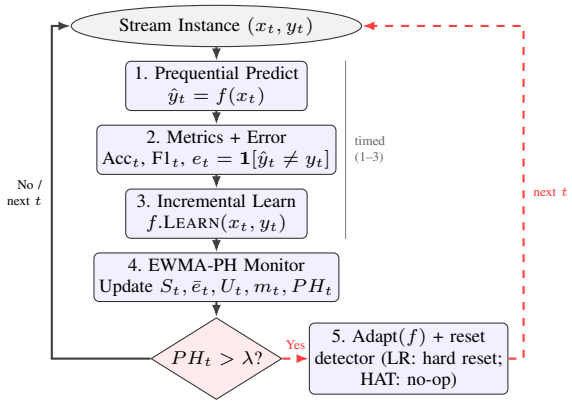



Fig. 1. Prequential pipeline as implemented in `evaluate()`. The latency timer covers stages 1–3; detector update and adaptation (stages 4–5) run after the model has learned $(x_t, y_t)$.

**1. Bias-Corrected EWMA Baseline:** A fading accumulator $S_t$ tracks the historical error rate:

$$S_t = \alpha\, S_{t-1} + (1 - \alpha)\, e_t, \quad S_0 = 0 \tag{1}$$

where $\alpha \in (0, 1]$ is the fading factor. To remove the start-up bias of a raw exponential average when $t$ is small, the baseline is bias-corrected (the power $\alpha^t$ is maintained incrementally):

$$\bar{e}_t = \frac{S_t}{1 - \alpha^t} \tag{2}$$

For $t \ll 1/(1-\alpha)$, $\bar{e}_t$ approximates the cumulative arithmetic mean of the observations; as $t$ grows, $\alpha^t \to 0$ and $\bar{e}_t$ relaxes into an exponential moving average with an effective memory of about $1/(1-\alpha)$ samples. This lets the baseline follow slow changes in the error rate without being dominated by isolated early errors [6].

**2. Cumulative-Sum Statistic:** The cumulative sum $U_t$ accumulates the deviation of each error from the baseline, offset by a tolerance $\delta$:

$$U_t = U_{t-1} + \big(e_t - \bar{e}_t - \delta\big), \quad U_0 = 0 \tag{3}$$

Under stationarity $\mathbb{E}[e_t - \bar{e}_t - \delta] \approx -\delta < 0$, so $U_t$ behaves like a random walk with a slight downward drift and noise does not accumulate into an alarm.

**3. Test Statistic and Drift Decision:** The detector records the running minimum of $U_t$,

$$m_t = \min(m_{t-1}, U_t), \quad m_0 = 0, \tag{4}$$

and the Page-Hinkley statistic measures the rise of $U_t$ above it:

$$PH_t = U_t - m_t. \tag{5}$$

When the error rate increases abruptly, $\bar{e}_t$ lags behind, the increments $(e_t - \bar{e}_t - \delta)$ become persistently positive, and $U_t$

climbs away from its frozen minimum. Drift is signalled as soon as

$$PH_t > \lambda. \tag{6}$$

The test is one-sided: it reacts only to increases in the error rate, which is the direction that matters for model degradation. If the error rate jumps from a baseline $\bar{e}$ to $p$, the statistic grows by roughly $p-\bar{e}-\delta$ per sample, so the expected detection delay is approximately $\lambda/(p-\bar{e}-\delta)$ samples. After an alarm, `reset()` reinitializes the state in place ($n, S_t, \bar{e}_t, U_t, m_t, PH_t \leftarrow 0$ and $\alpha^t \leftarrow 1$), so monitoring restarts from a fresh baseline. No warm-up period is used. Each update costs a constant number of arithmetic operations, and the detector stores seven scalar state variables ($n, S_t, \alpha^t, \bar{e}_t, U_t, m_t, PH_t$) plus its three hyperparameters $(\alpha, \delta, \lambda)$.

### C. Model Selection Justification

We evaluate two contrasting incremental classifiers, both with `river`'s default hyperparameters:

- **Hoeffding Adaptive Tree (HAT):** `river`'s `HoeffdingAdaptiveTreeClassifier` grows splits using Hoeffding bounds (grace period 200, split confidence $10^{-7}$) and monitors each node with an ADWIN detector; when a node's error increases, an alternate subtree is grown and replaces the original if it becomes significantly more accurate. HAT therefore adapts on its own and can represent non-linear boundaries. By default it uses bootstrap sampling with no fixed seed, so its runs are not bit-for-bit repeatable.
- **Streaming Logistic Regression (LR):** a `river` pipeline `StandardScaler` $\rightarrow$ `LogisticRegression`, trained by SGD (learning rate 0.01, log loss, no regularization). It costs $O(d)$ per sample for $d$ features and has no built-in forgetting mechanism. It is fully deterministic.

The pairing of HAT and LR exposes the trade-off between representational capacity and throughput, and between internal (passive) and detector-triggered (active) adaptation. We use an EWMA-based reference level rather than the plain cumulative mean of the classic Page-Hinkley test because a cumulative mean becomes increasingly insensitive as $t$ grows [3], while sliding-window approaches must buffer past observations [1]; the EWMA baseline adapts its reference level in $O(1)$ space and time [6].

### D. Online Adaptation Protocol

On an alarm ($PH_t > \lambda$), the model-specific callback is executed:

- **Logistic Regression pipeline** (`reset_logreg_pipeline`): SGD-trained weights and the scaler's running statistics retain the old concept indefinitely, so both pipeline steps are replaced with freshly initialized `StandardScaler` and `LogisticRegression` instances (a *hard reset*).
- **Hoeffding Adaptive Tree** (`reset_hat`): the callback is deliberately a no-op. HAT keeps its structure and adapts only through its internal ADWIN-driven subtree replacement; for HAT the external detector acts purely as a monitor, and HAT serves as the internal-adaptation reference.

### E. Evaluation Metrics Formulation

The following quantities are computed from each run:

- **Rolling accuracy ($\text{Acc}_t$):** accuracy over the most recent $W = 500$ predictions (`utils.Rolling(metrics.Accuracy, 500)`), logged after every sample: $\text{Acc}_t = \frac{1}{\min(t,W)} \sum_{i=\max(1,t-W+1)}^{t} \mathbf{1}[\hat{y}_i = y_i]$.
- **Final rolling accuracy:** $\text{Acc}_N$, i.e. the accuracy over the last 500 samples. Because it covers only 500 predictions, it is sensitive to the local state of the stream at its end.
- **Overall prequential accuracy:** $\frac{1}{N} \sum_{t=1}^{N} \mathbf{1}[\hat{y}_t = y_t]$, the accuracy over the entire stream.
- **F1 score:** `river.metrics.F1`, the cumulative binary F1 score of the positive class (True: $f_1 + f_2 > \theta$ for SEA, UP for Elec2).
- **Detection delay ($D$):** $D = t_{\text{alarm}} - t^*$ for the injected change at $t^* = 25{,}000$ (SEA only).
- **Recovery (SEA only):** the post-change trough of $\text{Acc}_t$; the first sample after the trough at which $\text{Acc}_t \geq 0.95$ again; and the sustained-recovery point, after which $\text{Acc}_t$ never falls below 0.95 again. Each is reported as an offset from $t^*$.
- **Per-sample latency:** wall-clock time measured with `time.perf_counter_ns` around prediction, the two metric updates and learning (stages 1–3); we report the mean, median, 95th and 99th percentiles in microseconds. The detector update, the adaptation callback and logging are outside the timed region.
- **Alarm count ($K$):** the number of drift alarms over the stream.

Algorithm 1 summarizes the procedure.

**Algorithm 1** EWMA-PH Prequential Evaluation and Online Adaptation

**Input:** Data stream $\mathcal{S}$, model $f$, detector parameters $(\alpha, \delta, \lambda)$, adaptation function ADAPT

Detector state: $n \leftarrow 0,\ a \leftarrow 1,\ S \leftarrow 0,\ U \leftarrow 0,\ m \leftarrow 0$
**for** each instance $(x_t, y_t) \in \mathcal{S}$ **do**
  $t_{\text{start}} \leftarrow$ PERFCOUNTERNS()
  $\hat{y}_t \leftarrow f$.PREDICT$(x_t)$
  Update $\text{Acc}_t$ (window 500) and F1 with $(y_t, \hat{y}_t)$
  $e_t \leftarrow \mathbf{1}[\hat{y}_t \neq y_t]$
  $f$.LEARN$(x_t, y_t)$
  $\Delta t_t \leftarrow$ PERFCOUNTERNS() $- t_{\text{start}}$
  $n \leftarrow n+1;\ a \leftarrow \alpha\, a\ \{a = \alpha^n\}$
  $S \leftarrow \alpha S + (1-\alpha) e_t;\ \bar{e} \leftarrow S/(1-a)$
  $U \leftarrow U + (e_t - \bar{e} - \delta);\ m \leftarrow \min(m, U);\ PH \leftarrow U - m$
  **if** $PH > \lambda$ **then**
    Record alarm at $t$; ADAPT$(f)$ {LR: hard reset; HAT: no-op}
    Reset detector: $n, S, \bar{e}, U, m, PH \leftarrow 0;\ a \leftarrow 1$
  **end if**
  Log $(t, \text{Acc}_t, \Delta t_t, \text{alarm flag})$
**end for**

## VIII. Implementation Details

### A. Software

The framework is written in Python 3 on top of `river`, which provides the streaming learners, metrics, and dataset loaders; figures are produced with `matplotlib` (`seaborn-v0_8-paper` style). Telemetry is written and

read with Python's standard `csv` module; the project code does not use `numpy` or `pandas` directly. The Page-Hinkley detector is implemented from first principles in `page_hinkley.py` and does not depend on `river.drift`.

### B. Repository Structure

The repository (`Project_FML`) contains the following scripts:

- `page_hinkley.py`: the $O(1)$ EWMA-PH detector (1)–(6), with a self-test on a synthetic Bernoulli error stream whose rate jumps from 0.05 to 0.45.
- `stream_evaluation.py`: the prequential `evaluate()` loop (rolling accuracy, F1, per-sample latency, drift monitoring, adaptation hooks) and the two adaptation callbacks.
- `river_benchmarks.py`: the benchmark driver; builds the SEA stream with the numerically stable concept-mixing generator, loads Elec2, runs both models, and exports per-sample telemetry to `telemetry_sea.csv` and `telemetry_elec2.csv` (columns: sample index, model, rolling accuracy, latency in $\mu$s, drift flag).
- `plot_results.py`: renders the recovery curve (`figure1_recovery.png`) and the latency–accuracy plot (`figure2_pareto.png`) from the telemetry.
- `analyze_telemetry.py`: recomputes the derived metrics in Section IX from the telemetry (see Section VIII-F).
- `stream_dataset.py`, `make_sample.py`: a row-by-row CSV streaming generator with automatic numeric casting, and a script that writes a 20-row random CSV (`sample_stream.csv`). Running `stream_evaluation.py` directly on this file is a smoke test of the pipeline (`telemetry_results.csv`); it is not part of the reported results.

### C. Execution Environment

All reported runs and timings were produced on the authors' Apple-silicon Mac under macOS, in a single Python process, with each model evaluated sequentially. The logged latencies are integer multiples of 41.67 ns, the resolution of the 24 MHz system timer on Apple-silicon machines; this is more than two orders of magnitude finer than the measured per-sample latencies. Absolute latencies depend on the host CPU and Python runtime, so the latency ratios between models are more transferable than the absolute values.

### D. Hyperparameter Settings

- **EWMA-PH detector:** fading factor $\alpha = 0.9999$ (effective memory $1/(1-\alpha) = 10{,}000$ samples), tolerance $\delta = 0.005$, threshold $\lambda = 50$. These are the conventional Page-Hinkley settings (the defaults of `river`'s own `PageHinkley`); they were fixed a priori and not tuned on either stream. One fresh detector instance is used per model and dataset.
- **Evaluation:** rolling window $W = 500$; telemetry is written for every sample, and a progress summary is printed every 100 samples.
- **SEA stream:** variants 0 ($\theta = 8$) and 1 ($\theta = 9$), seed 42 for both generators and for the mixing, change position 25,000, width 1, $N = 50{,}000$.
- **Classifiers:** `river` defaults for `HoeffdingAdaptiveTreeClassifier` (no seed) and for `LogisticRegression` (SGD, learning rate 0.01).

### E. Training, Validation, and Testing Procedure

Static protocols ($k$-fold cross-validation or fixed splits) are not meaningful for evolving streams, so we use the prequential protocol: every instance first tests the current model and only then trains it, so every prediction is made on an unseen instance in a single pass. No separate validation split was used, and no hyperparameter was tuned on the evaluation streams.

### F. Telemetry Verification and Reproducibility

Since the rolling window size is known, the per-sample correctness indicator $c_t$ can be recovered exactly from the logged rolling accuracy ($c_t = \min(t, W)\,\mathrm{Acc}_t - \sum_{i=\max(1,t-W+1)}^{t-1} c_i$). `analyze_telemetry.py` performs this reconstruction for all four runs, recomputes overall accuracy, F1 (using the dataset labels) and recovery metrics, and replays the detector offline on $e_t = 1 - c_t$. The replay reproduces all 22 logged alarms (1 + 1 on SEA, 10 + 10 on Elec2) at exactly the logged sample indices, confirming that the telemetry and the detector implementation are consistent.

The SEA stream and the LR pipeline are deterministic: an independent re-run of the SEA benchmark (`river` 0.26.1, Python 3.13, Linux) reproduced the LR rolling-accuracy trace identically at all 50,000 samples, including the alarm at sample 26,356. HAT is not seeded, so its results vary between runs: in five such re-runs it again raised exactly one alarm, with delays between 1,056 and 1,356 samples and overall prequential accuracy between 0.967 and 0.971 (0.971 in the reported run). Latencies from these re-runs are not used anywhere in this paper.

## IX. Experimental Results and Performance Evaluation

Tables II and III summarize accuracy, detection, and latency for the four runs; Table IV details the SEA change.

TABLE II
Accuracy and Drift Detection

| Dataset | Model | Overall Acc. (all $N$) | Final Acc. (last 500) | F1 | Alarms | Delay |
|---|---|---|---|---|---|---|
| SEA (50k) | HAT | 0.971 | 0.988 | 0.978 | 1 | 1,213 |
| SEA (50k) | Scaler+LR | 0.993 | 0.998 | 0.994 | 1 | 1,356 |
| Elec2 (45k) | HAT | 0.840 | 0.794 | 0.807 | 10 | N/A |
| Elec2 (45k) | Scaler+LR | 0.837 | 0.912 | 0.799 | 10 | N/A |

### A. Evaluation on the Synthetic SEA Stream

Before the change, LR reaches a rolling accuracy close to 1.0 after a short warm-up (accuracy 0.994 over samples 1–25,000), while HAT is somewhat lower and noisier (0.971; its rolling accuracy dips as low as 0.936 after warm-up). Neither model

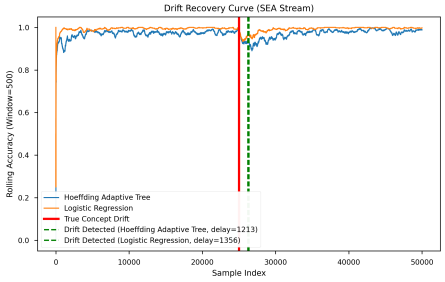



Fig. 2. Rolling accuracy ($W = 500$) on the SEA stream. The red line marks the concept change at sample 25,000; the dashed green lines mark the detector alarms for HAT (sample 26,213, delay 1,213) and LR (sample 26,356, delay 1,356). The LR model is hard-reset at its alarm; HAT is not modified by its alarm.

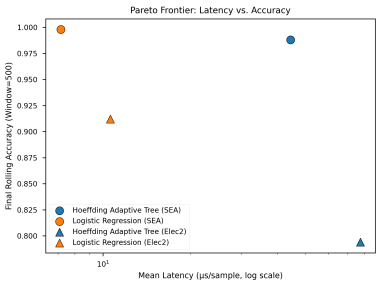



Fig. 3. Mean per-sample latency (log scale) versus final rolling accuracy (last 500 samples) as produced by `plot_results.py`. Circles: SEA; triangles: Elec2; orange: Scaler+LR; blue: HAT. Overall prequential accuracies are given in Table II.

TABLE III
PER-SAMPLE LATENCY IN $\mu$S (PREDICT + METRIC UPDATES + LEARN)

| Dataset | Model | Mean | Median | p95 | p99 | Samples/s |
|---|---|---|---|---|---|---|
| SEA | HAT | 44.40 | 39.83 | 61.71 | 103.54 | 22,500 |
| SEA | Scaler+LR | 7.15 | 7.04 | 7.33 | 9.04 | 139,900 |
| Elec2 | HAT | 77.32 | 63.50 | 167.00 | 320.83 | 12,900 |
| Elec2 | Scaler+LR | 10.61 | 10.50 | 11.08 | 12.92 | 94,300 |

Samples/s $= 10^6/$mean; it excludes the detector update and loop overhead and is therefore an upper bound on end-to-end throughput.

TABLE IV
SEA CONCEPT CHANGE ($t^* = 25{,}000$): DETECTION AND RECOVERY

| | HAT | Scaler+LR |
|---|---|---|
| Accuracy, samples 1–25,000 | 0.971 | 0.994 |
| Max. $PH_t$, samples 1–25,000 ($\lambda = 50$) | 20.8 | 6.0 |
| Error rate between change and alarm | 0.074 | 0.049 |
| Detection delay $D$ | 1,213 | 1,356 |
| Rolling-accuracy trough (sample) | 0.890 (26,770) | 0.934 (25,508) |
| First return to $\text{Acc}_t \geq 0.95$ | $t^* + 2{,}823$ | $t^* + 953$ |
| Sustained $\text{Acc}_t \geq 0.95$ from | $t^* + 6{,}519$ | $t^* + 1{,}837$ |
| Accuracy, samples 25,001–50,000 | 0.971 | 0.992 |

triggers a false alarm in the pre-change phase: the largest Page-Hinkley statistic before sample 25,000 is 20.8 for HAT and 6.0 for LR, well below $\lambda = 50$. After the change at sample 25,000 (Fig. 2, Table IV):

- **Detection:** Each model receives exactly one alarm: HAT at sample 26,213 (delay 1,213) and LR at sample 26,356 (delay 1,356); no further alarm occurs for the rest of the stream. The delays agree with the approximation $\lambda/(p - \bar{e} - \delta)$: the error rate between change and alarm was 7.4% for HAT and 4.9% for LR, about 4–5 percentage points above each model's pre-change error rate. LR's error rate rose less, and its alarm came later, because SGD had already started to move its decision boundary towards the new concept.
- **Adaptation without the alarm:** Both post-change error rates are below the 8.4% that a model frozen at the old boundary would incur, so both learners were already adapting before any alarm. LR's rolling accuracy returned to 0.95 at $t^* + 953$, *before* its alarm at $t^* + 1{,}356$.
- **Effect of the LR hard reset:** The reset therefore replaced a model that had already partly re-converged. Accuracy over the 500 samples after the reset was 0.952, compared with 0.970 over the 500 samples before it, then rose to 0.968 and 0.983 over the following 500 and 1,000 samples. LR's rolling accuracy stays at or above 0.95 from $t^* + 1{,}837$ (481 samples after the reset) onward and reaches 0.998 at the end of the stream.
- **HAT recovery:** HAT's alarm does not change the model; its recovery relies on ADWIN-driven subtree replacement and continued tree growth. Its rolling accuracy falls to 0.890 at sample 26,770, first returns to 0.95 at $t^* + 2{,}823$, and remains above 0.95 only from $t^* + 6{,}519$; it ends the stream at 0.988.
- **Overall:** LR's overall prequential accuracy is 0.993 (F1 0.994) against 0.971 (F1 0.978) for HAT, and LR is more accurate both before and after the change.
- **Latency:** LR averages 7.15 $\mu$s per sample (median 7.04 $\mu$s), 6.2$\times$ faster than HAT's 44.40 $\mu$s (median 39.83 $\mu$s). LR's latency distribution is also much tighter (p99 9.04 vs. 103.54 $\mu$s).

### B. Evaluation on the Electricity Market Benchmark (Elec2)

On Elec2 (45,312 samples) both models raise 10 alarms (Table II).

- **Alarm positions:** LR: 2,472, 5,352, 15,689, 20,524, 22,782, 27,389, 31,366, 36,606, 43,334, 44,032. HAT: 6,236, 9,431, 14,961, 16,227, 19,327, 22,834, 24,522, 31,136, 36,548, 43,793. The positions differ because each detector monitors its own model's error stream. Intervals between consecutive alarms range from 698 to 10,337 samples for LR (median 4,607) and from 1,266 to 7,245 for HAT (median 3,507). Elec2 has no ground-truth drift points, so individual alarms cannot be classified as true or false.
- **Accuracy:** Over the whole stream, the two models are practically tied: overall prequential accuracy is 0.840 for HAT and 0.837 for LR, with F1 (UP class) of 0.807 and 0.799. Over nine consecutive 5,000-sample blocks, LR is more accurate in five and HAT in four, and LR's rolling accuracy exceeds HAT's at 50.0% of the samples. Both

rolling-accuracy curves fluctuate widely (HAT 0.63–0.98, LR 0.67–0.95).

- **Final rolling accuracy:** The final 500-sample window favours LR (0.912 vs. 0.794). This reflects the end of the stream rather than the whole run: LR's last alarm at sample 44,032 reset it 1,280 samples before the end, and over samples 44,033–45,312 LR reached 0.876 against 0.802 for HAT.
- **Latency:** LR averages 10.61 $\mu$s per sample (median 10.50 $\mu$s), 7.3$\times$ faster than HAT's 77.32 $\mu$s (median 63.50 $\mu$s); HAT's tail is heavy (p99 320.83 $\mu$s). Both models are slower than on SEA, plausibly because Elec2 has eight attributes rather than three; for HAT, the size of the tree and the number of ADWIN monitors also play a role.

### C. Latency–Accuracy Trade-off

Fig. 3 plots mean latency (log scale) against final rolling accuracy, as generated by `plot_results.py`. On SEA, LR dominates HAT on both axes, and this holds whichever accuracy measure is used (Table II). On Elec2, the apparent accuracy gap in Fig. 3 is specific to the last 500 samples; in overall prequential accuracy the two models differ by only 0.2 percentage points. LR's clear advantage on Elec2 is therefore throughput (7.3$\times$) at equal accuracy, not higher accuracy.

### D. Comparison with Baseline Models

HAT serves as the intrinsic adaptive baseline: it adapts with `river`'s native ADWIN-based alternate-subtree mechanism and is not modified by the external detector. The Scaler+LR pipeline represents explicit detection followed by a hard reset. The repository does not contain runs of other baselines—a static LR without resets, a Page-Hinkley test without the EWMA baseline, or alternative detectors such as DDM or ADWIN coupled to LR—so the contributions of the EWMA baseline and of the reset itself cannot yet be isolated. Adding these runs requires only new entries in `make_model_configs()` in `river_benchmarks.py` (e.g. a no-op adaptation callback for LR, or a different detector object).

## X. Discussion and Result Analysis

### A. Interpretation of Results

1) **Elec2 accuracy matches the original projection:** The project synopsis projected roughly equal accuracy for both models on Elec2 ($\approx 83.6\%$). The overall prequential accuracies (83.7% for LR and 84.0% for HAT) agree with this projection. The large difference in final rolling accuracy (91.2% vs. 79.4%) comes from a single 500-sample window and should not be read as LR outperforming HAT on Elec2.
2) **What resets do and do not show:** After an alarm, accuracy often rebounds, but this happens for HAT as well, although HAT's alarm leaves the model unchanged (e.g. from 0.738 to 0.844 around sample 22,834, and from 0.806 to 0.908 around sample 14,961, in the 500 samples before and after the alarm). Because alarms tend to fire at local accuracy troughs, a post-alarm rebound is not by itself evidence that a reset helped. On SEA, LR had largely re-adapted through SGD before its alarm, and the reset caused a short, shallow dip before the accuracy rose above its pre-reset level. Quantifying the benefit of resets requires the no-reset ablation described above.
3) **Linear model versus tree:** SEA's concepts are linear in $(f_1, f_2)$, so a linear model fits them almost exactly, while HAT must approximate the diagonal boundary with axis-parallel splits; this explains LR's higher accuracy before and after the change. On Elec2, where the boundary is not linear, the tree's extra capacity and LR's resets end up at the same overall accuracy.
4) **Latency:** LR is 6.2$\times$ (SEA) and 7.3$\times$ (Elec2) faster than HAT. A linear prediction and SGD update need one $O(d)$ dot product and vector update, whereas HAT traverses the tree, updates split statistics at the leaf, evaluates Hoeffding bounds, and updates ADWIN monitors along the path.
5) **Detection behaviour:** On SEA, the detector raised no false alarm in 25,000 stationary samples per model and a single alarm after the change, with a delay of 1.2–1.4 thousand samples for an error increase of 4–5 percentage points; larger error jumps are detected proportionally faster. On Elec2 it raised about one alarm per 4,500 samples per model, rather than firing continuously.
6) **Context on Elec2:** A trivial "no-change" classifier that predicts the previous label reaches 85.3% accuracy on Elec2, owing to strong autocorrelation of the labels [11]; both models remain slightly below this reference, so their absolute Elec2 accuracies should be read with that in mind.

### B. Strengths

- **Constant resource cost of detection:** Each detector update is a fixed number of arithmetic operations on seven scalar state variables, with no buffering of past observations.
- **High throughput:** The Scaler+LR pipeline sustains on the order of 94,000–140,000 predict-and-learn cycles per second on a single core (Table III), 6–7$\times$ more than HAT, with a tight latency distribution.
- **No false alarms on the stationary SEA segment:** The maximum statistic before the change stayed at 6.0–20.8 against a threshold of 50, while the abrupt change was detected for both models.
- **Verifiable results:** All alarms reproduce exactly from the logged telemetry, and the LR run reproduces bit-for-bit in an independent re-run on different hardware.

### C. Limitations

- **Detection delay:** With $\lambda = 50$, a moderate error increase (4–5 percentage points) takes more than 1,200 samples to detect; on SEA, LR had already recovered through SGD before its alarm.
- **Missing ablations:** No runs exist for LR without resets, for a plain (non-EWMA) Page-Hinkley test, or for other detectors, so the specific contributions of the EWMA baseline and of the hard reset are not isolated.

- **Information loss from hard resets:** A reset discards all learned weights and scaler statistics, producing a cold-start dip (observed on SEA) and making false alarms costly.
- **Benchmark diversity:** Only two binary streams were used, one with a single abrupt change. Gradual, recurring, multi-class, and high-dimensional streams remain untested, and Elec2 offers no ground-truth drift points.
- **Single run and unseeded HAT:** Each configuration was run once. HAT is unseeded, so its alarm positions and accuracy vary between runs (Section VIII-F).
- **Sensitivity to $\alpha$, $\delta$, $\lambda$:** The conventional defaults were used without tuning; streams with frequent drift may need different values [6].
- **Measurement scope:** The reported latencies exclude the detector update and loop overhead and come from one machine; memory consumption (e.g. HAT's tree growth) was not profiled.

### *D. Connection to Research Gap and Objectives*

- **Objective 1 (constant-time detector):** Fulfilled. The detector runs in $O(1)$ time and memory on the error stream without windows and reproduces all logged alarms on replay.
- **Objective 2 (adaptive EWMA baseline):** Fulfilled in implementation. The bias-corrected EWMA baseline produced no false alarm on the stationary SEA segment and a timely alarm after the change; because no fixed-baseline Page-Hinkley run exists, the share of this behaviour attributable to the EWMA baseline is not yet quantified.
- **Objective 3 (coupling with incremental learners):** Fulfilled. Both learners run inside the prequential adaptation pipeline: LR with a hard reset on every alarm, HAT with its internal adaptation and the external detector as a monitor.
- **Objective 4 (prequential evaluation):** Fulfilled. Detection delay (1,213 and 1,356 samples), recovery (sustained $\geq 0.95$ from $t^* + 1{,}837$ for LR and $t^* + 6{,}519$ for HAT), accuracy (overall 0.993/0.971 on SEA, 0.837/0.840 on Elec2), and latency (7.15/44.40 $\mu$s and 10.61/77.32 $\mu$s) were measured.

## XI. Conclusion and Future Scope

We developed and evaluated a lightweight concept drift detection and online adaptation framework for streaming classification, built around a Page-Hinkley test with a bias-corrected EWMA baseline that runs in $O(1)$ time and memory per sample. On the SEA stream with an abrupt change, the detector raised exactly one alarm per model after delays of 1,213 (HAT) and 1,356 (LR) samples and none in the stationary segment; the streaming LR pipeline reached 99.3% overall accuracy and stayed at or above 0.95 rolling accuracy from 1,837 samples after the change, against 97.1% and 6,519 samples for HAT. On Elec2, LR with hard resets matched HAT's overall accuracy (83.7% vs. 84.0%) while processing each sample 7.3$\times$ faster (10.61 vs. 77.32 $\mu$s). A fast linear learner combined with a constant-time drift detector is therefore a competitive and far cheaper alternative to an adaptive tree on these streams, although the specific benefit of the hard reset over plain incremental learning remains to be isolated.

Future work will focus on:

1) **Ablation baselines:** LR without resets, a fixed-baseline Page-Hinkley test, and ADWIN/DDM-triggered adaptation, run over multiple seeds.
2) **Soft and ensemble adaptation:** weight decay, warm-started replacements, or ensemble blending instead of hard resets, to avoid cold-start dips during gradual drift.
3) **Faster detection:** self-tuning of $\alpha$, $\delta$, and $\lambda$ [6], and proactive drift anticipation [8], [9].
4) **Broader benchmarks:** gradual and recurring synthetic drifts with known change points, multi-class and high-dimensional streams.
5) **Resource profiling:** end-to-end throughput including the detector, and memory profiling of the learners.

## Use of AI Tools

In accordance with academic integrity guidelines, AI-based tools were used during the preparation of this report for LaTeX typesetting, alignment against the evaluation rubric, grammatical polishing, and auditing the manuscript against the repository's code and telemetry, including drafting `analyze_telemetry.py`, which recomputes the derived metrics from the logged telemetry. The framework implementation, benchmark executions, and telemetry data were produced by the authors, and all reported numbers can be regenerated from the repository.

## References


[1] D. Mulimani, P. Patil, S. Totad, and R. Benni, "Online Detection and Adaptation of Concept Drift in Streaming Data Classification," *Procedia Computer Science*, vol. 235, pp. 2803–2811, 2024.

[2] P. Wu and N. Xiong, "Online Learning of Bayesian Classifiers with Nonstationary Data Streams," *International Journal of Network Dynamics and Intelligence*, vol. 2, no. 3, 2023.

[3] S. Parasteh and S. Sadaoui, "A Probabilistic Approach for Detecting Real Concept Drift," *Proceedings of the 16th International Conference on Agents and Artificial Intelligence (ICAART)*, vol. 2, pp. 301–311, 2024.

[4] K. Wan, Y. Liang, and S. Yoon, "Online Drift Detection with Maximum Concept Discrepancy," *Proceedings of the 30th ACM SIGKDD Conference*, pp. 2924–2935, 2024.

[5] G. J. Aguiar and A. Cano, "Enhancing Concept Drift Detection in Drifting and Imbalanced Data Streams through Meta-Learning," *Virginia Commonwealth University*, 2023.

[6] S. N. Mohd Rum and Q. Song, "An EWMA-Based Adaptive Thresholding Concept for Autoencoder-Based Concept Drift Detection in Data Streams," *International Journal of Advanced Computer Science and Applications*, vol. 17, no. 5, 2026.

[7] L. Chambers, M. M. Gaber, and H. Ghomeshi, "AdaDeepStream: streaming adaptation to concept evolution in deep neural networks," *Applied Intelligence*, vol. 53, pp. 27323–27343, 2023.

[8] J. V. Guerrero Cano, G. J. Aguiar, and A. Cano, "Anticipating to Change: A Proactive Approach for Concept Drift Adaptation in Data Streams," *Machine Learning*, vol. 115, no. 3, 2026.

[9] L. Zhao and Y. Shen, "Proactive Model Adaptation Against Concept Drift for Online Time Series Forecasting," *Proceedings of the 31st ACM SIGKDD Conference*, 2025.

[10] C. Fellicious, J. Mitrović, L. Wendlinger, M. Gancarski, and M. Granitzer, "SUDS: A Strategy for Unsupervised Drift Sampling," *arXiv preprint arXiv:2411.02995*, 2024.

[11] I. Žliobaitė, "How good is the Electricity benchmark for evaluating concept drift adaptation," *arXiv preprint arXiv:1301.3524*, 2013.